Design: multi-product scans

Issue **chukssomzzy/deepertones_be#74** · written 2026-10-18 · as-built traced on `origin/staging` at `d7c6615` · #55 read from its four open PRs (#61, #64, #72, #73), not merged code

AS-BUILT: on staging, every claim traced to a file and line

IN FLIGHT: #55, open PRs, not on staging

AS-DESIGNED: proposal, nothing here exists yet

> **The change in one sentence:** a product-scan photo becomes its own record, a **scan photo** (proposed). Recognising it can turn its one placeholder item into up to five items, one per readable product. Each item then runs the existing per-item pipeline unchanged. The photo is deleted once none of its items still shows it.

1. Where the issue and the code disagree

The issue was written from a sketch. Where it disagrees with the code, the code is what's true today. Items 1 to 4 change what the feature can promise. Items 5 to 10 are gaps the issue doesn't cover.

1. No stock photo exists at confirmation time. The issue says people confirm each product "by its catalog stock photo". But an item reaches `needs_confirmation` in `recognise`, before `match` gives it a product (scans.service.ts:561–575, 602–643). Stock images are imported only after research finds and stores ingredients (catalog.service.ts:278–280, 508–528). And `itemView` looks up a stock image only for a `ready` item with a formulation (scans.service.ts:853–854). So at confirmation the only image is the user's own photo, through a signed link (:858–861). For a product the catalog has never seen, brand and name are the normal case, not the fallback. Showing "stock image when matched" also needs a new lookup by product id, because today the lookup is by formulation and for ready items only.

2. Today, front photos are deleted only on rejection. The only place a front photo is deleted is in `recognise`, for a photo that is rejected (scans.service.ts:580). Items that end `ready` or `failed` keep `photo_key` until the guest cleanup runs. `labelUnreadable` even says so: "keeping its front photo like the other failures" (:724–725). `itemView` then shows that photo for any item with a key and no stock image (:858). The new rule ("delete once no item needs it") therefore **removes the image a ready product with no stock photo shows today**, single-photo scans included. That is a change in behaviour, not just the old rule widened.

3. The daily-limit check moves from the request into a queue job. `assertWithinQuota` runs only inside `create` and `addBackPhoto`. It takes the user from the request context and the IP from the request header (scans.service.ts:174, 181–182, 306–308, 920–934). Skipping extra products as `daily_limit` has to happen inside `recognise`, which runs as a system actor with no request. So the user comes from the item, and the IP hash comes from the `scans` row (scans.table.ts, `ip_hash`). The issue mentions only the user allowance, but the per-IP limit applies to each product too.

4. Retrying recognition loses or duplicates products once a photo can yield several. When the move out of `recognising` is undone, `earlierRead` rebuilds the earlier answer from the item's own columns, so the retry doesn't pay the model twice (scans.service.ts:533, 585–600, 807–809). Those columns hold one product. With several products, a retried `recognise` would either drop the extra products or create them twice. The issue doesn't cover this. §4.2 proposes keeping the whole answer on the scan photo, and inserting the extra items in the same D1 batch as the placeholder's guarded move.

5. Item positions can collide. `(scan_id, position)` is unique (scans.table.ts:72). The `recognise` jobs for different photos of one scan run at the same time, so "next position = max + 1" would collide. §4.2 proposes a deterministic slot instead.

6. Retakes (#61) assume each item owns its front photo. #61's `retake()` writes the new photo to the item's own `photo_key`. The issue says items reference a shared photo, and also that "a retake gives the item its own single-product photo". So an item needs **both** a nullable `photo_id` and its own nullable `photo_key`. Separately, story 15 ("retake one product when its read was wrong") had no route: #61 allows a retake only from `needs-retake`, the state for an unreadable photo. **Settled (B):** story 15 is dropped, and a wrong read is fixed through confirm or #72's typed name.

7. "Rejected exactly as today" won't match once #55 lands. The issue says a photo with no readable product rejects its placeholder as `unreadable`. After #61, an unreadable photo goes to `needs-retake`, or to `needs-identification` on the last try, not to `rejected`. For a photo of several products, a retake would swap a shelf photo for a single-product one. **Settled (A):** the server can't tell a shelf photo from a single-product one, so the rule goes by what was seen. No readable product and exactly one unreadable product → #55's `needs-retake`. Anything else → `rejected`, with the photo's counts.

8. The guest cleanup reads only item keys. `photoKeysForUsers` collects `scan_items.photo_key` and `back_photo_key` (scans.repository.ts:163–176), and `ScanDataCleaner` deletes those (scans.cleaner.ts:21–25). Keys stored in the new table would be left behind. Story 34 covers photos missed by the reaper, but not this path.

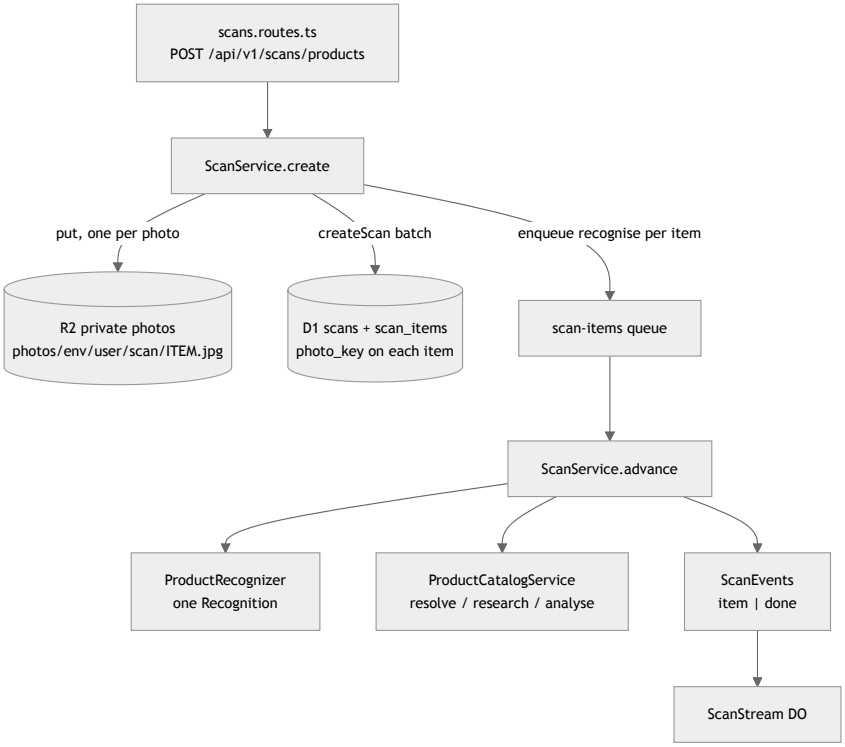
9. The stream can't carry a third event type. `ScanEvent` is `item | done` and `ScanSnapshot.items` is flat (scan-events.provider.ts:7–14). `ScanStream.publish` handles `done` and frames **everything else** as an item (scan-stream.ts:61–68). So a `photo` event needs its own branch in the Durable Object, and `catchUp` has to compare photos as well as items (scans.service.ts:245–269).

10. A face has no skip reason. The issue has the recognizer count "faces and people", but its list of skip reasons (`not-a-beauty-product`, `unreadable`, `over-limit`, `daily-limit`) has no slot for them. **Settled (E):** faces aren't counted, and the recognizer doesn't report them. A photo with only a face still gets today's top-level `face-or-person` classification.

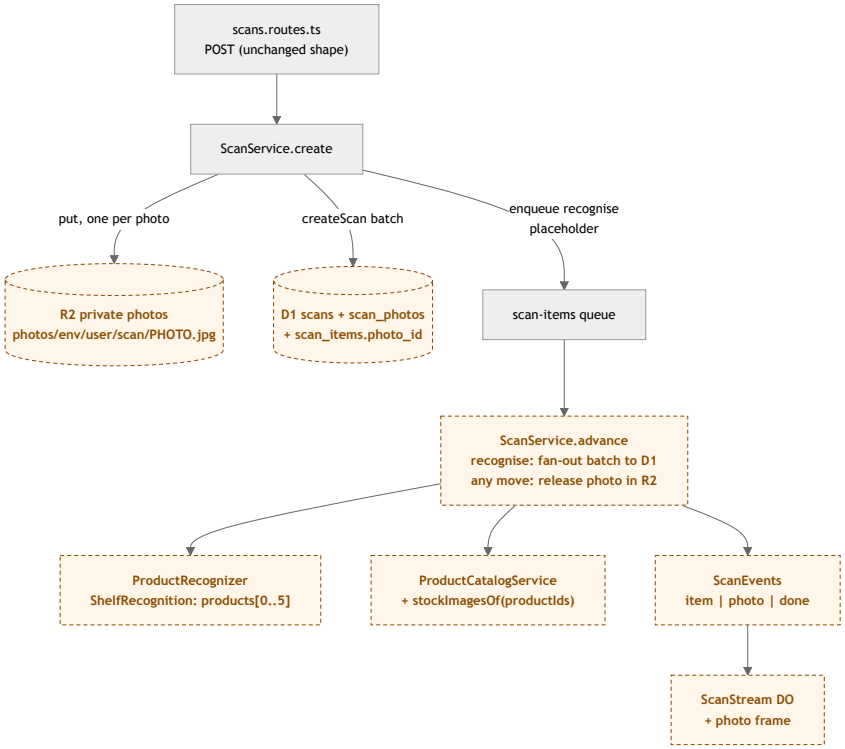
2. Module map

Only the scans module and two providers change shape. The catalog gains one read. Every other module is untouched.

AS-BUILT Where an upload goes today: one photo = one R2 object = one `scan_items` row.

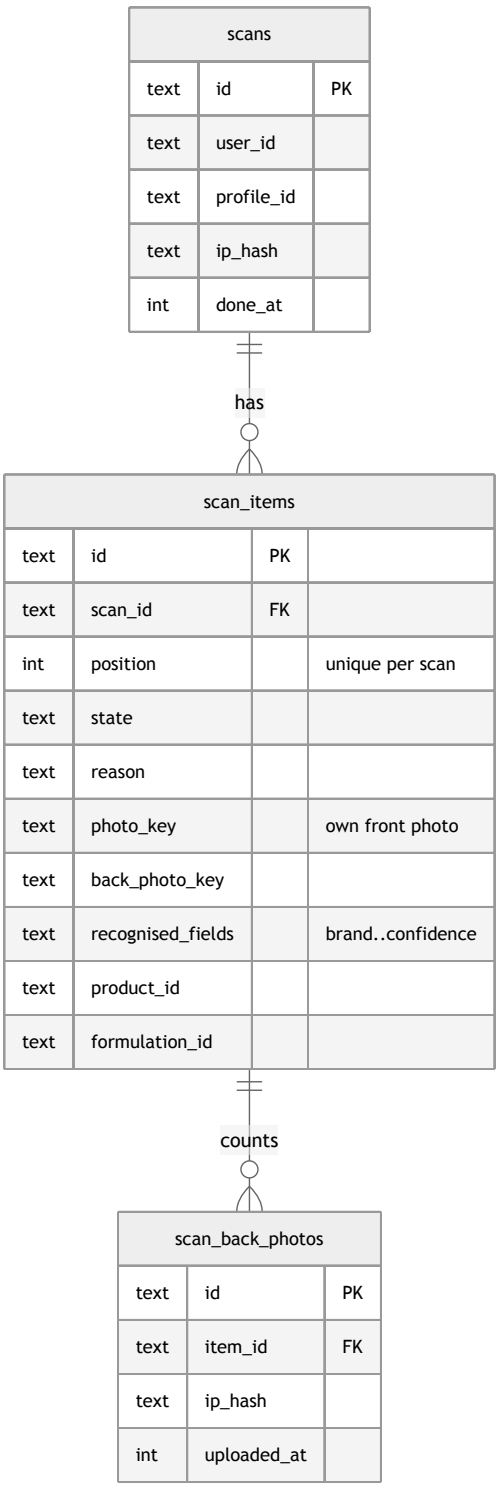


AS-DESIGNED Changed parts in amber. One photo becomes one **scan_photos** row (proposed), which recognition turns into 1 to 5 items.

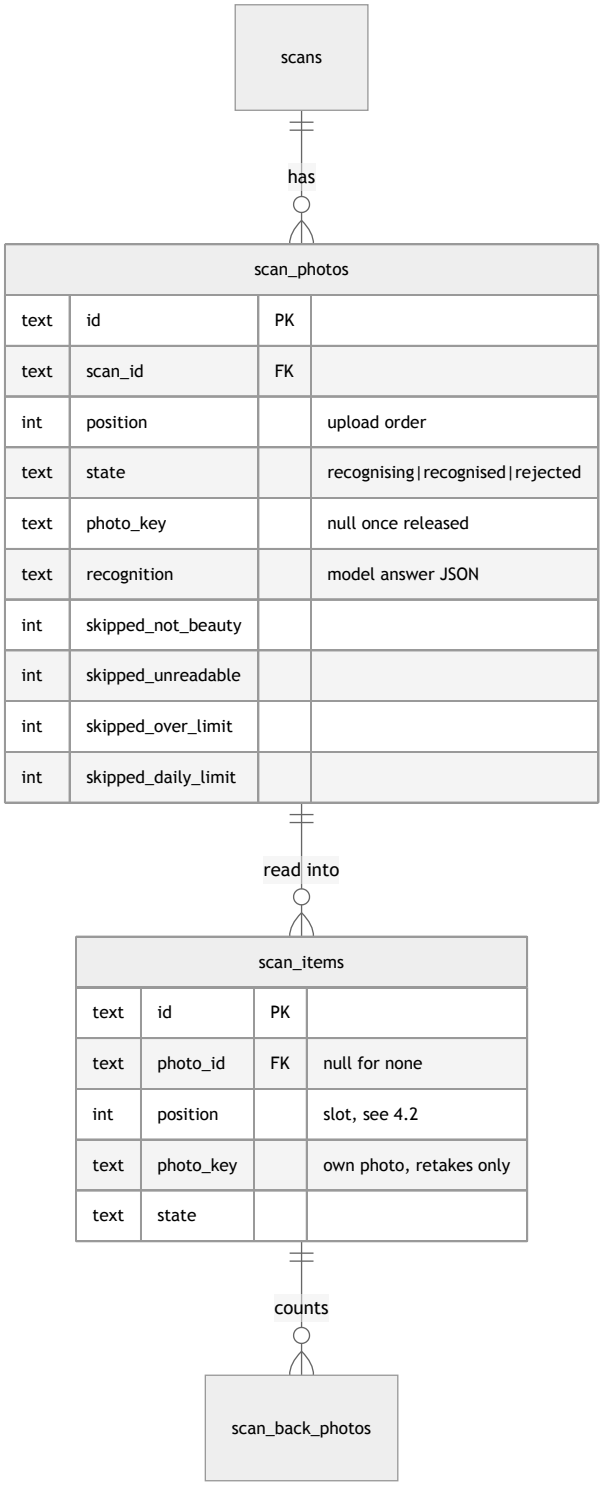


3. Data

AS-BUILT scans.table.ts:11–101



AS-DESIGNED **scan_photos** (proposed) and two new item columns.



AS-DESIGNED

- `scan_items.photo_key` **stays, but only for an item's own photo, which means a #61 retake.** A new `photo_id` points at the shared photo. The item's image is its own key if it has one, else the photo's key, else nothing.
- `scan_photos.recognition` stores the model's whole answer, written in the fan-out batch. A retried `recognise` reads it instead of paying again. It replaces `earlierRead` for photos, and `earlierRead` stays for retakes.
- **The migration is mechanical (pre-production).** It creates `scan_photos`, adds `photo_id`, and inserts one photo per existing item, copying its key. The number is whatever comes after #55's migrations once they're renumbered (see §7).

4. Flows, before and after

4.1 Upload: `POST /api/v1/scans/products`

AS-BUILT

```
scans.routes.ts:116  createScan          IP limiter → signedIn → user limiter → bodyLimit (:28–33)
└─ ScanService.create(photos, clientIp)   scans.service.ts:173
   ├─ 1..10 photos, checkPhoto each      :175–179  JPEG/WebP, ≤2 MB, ≤1600 px
   ├─ assertWithinQuota(user, ip, N)     :182  N = photos; 30/user, 100/IP per 24 h
   ├─ per photo: item id + photoKey      :185–189  photos/env/user/scan/<itemId>.ext
   ├─ photos.put × N                     :191  R2 first, so no row points at nothing
   ├─ repository.createScan(scan, items) :192  one D1 batch
   └─ enqueue recognise per item         :198–208  failure → item failed/unavailable
→ 202 { id, createdAt, items[] }         schemas: "One per photo, in photo order" (:118)
```

AS-DESIGNED

```
createScan  route, limits and body: unchanged
└─ ScanService.create
   ├─ checkPhoto, assertWithinQuota(N)   unchanged: one per photo, its placeholder
   ├─ per photo: photo id + placeholder item id
   │    photoKey = photos/env/user/scan/<photoId>.ext
   ├─ photos.put × N
   ├─ repository.createScan(scan, photos, items)  one batch: scan, N photos, N items (photo_id set)
   └─ enqueue recognise per placeholder  unchanged
→ 202 { id, createdAt, photos[]: { id, position, state, skipped, items[] } }
```

4.2 Recognise: the fan-out

AS-BUILT

```
advanceScanItem job (scans.jobs.ts:5) → ScanService.advance(itemId,'recognise')  :352
└─ recognise(item)                                    :531
   ├─ earlierRead(item) ?? recognizer.recognize(photo) :533–541  reused after an undo
   │   ├─ AiBudgetExceededError   → failed/research-busy        :543–546
   │   ├─ AnthropicRequestError   → failed/unavailable, Permanent :549–552
   │   └─ AnthropicRefusalError   → treated as unreadable        :556
   ├─ beauty-product:
   │   └─ transition → matching (+ enqueue match) | needs-confirmation (+ push)  :560–576
   └─ otherwise: photos.delete(photoKey), then → rejected(reason), photo_key NULL  :579–582
transition(): guarded UPDATE → enqueue next → publish item (+ done if all final)  :789–832
```

Recognizer: `Recognition` is one product or one of three classifications (product-recognizer.provider.ts:4–12). The Anthropic adapter's schema is a single object, and anything out of bounds becomes `unreadable` (anthropic.product-recognizer.ts:35–44, 85–101).

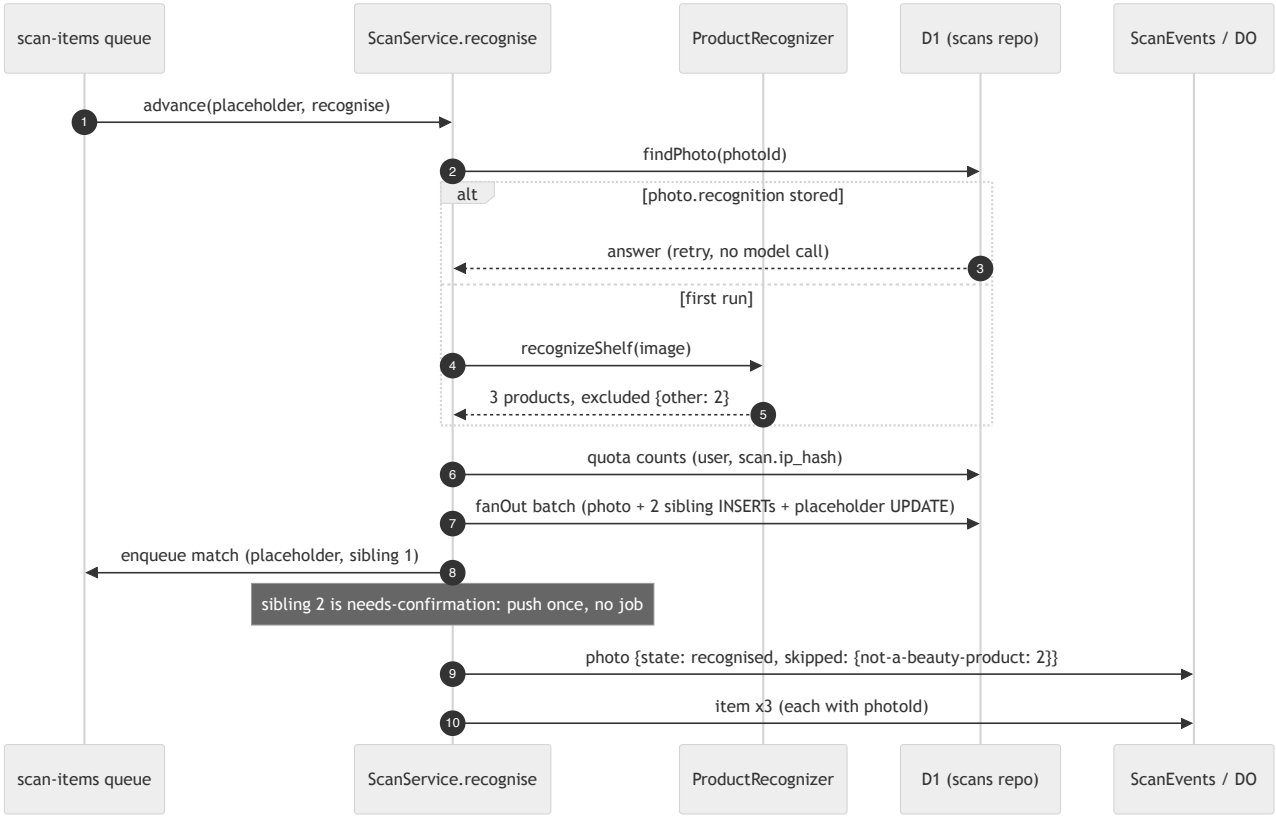
AS-DESIGNED

Recognizer contract (proposed): `recognizeShelf(image) → ShelfRecognition`. It keeps today's top-level `classification`, so a photo with no readable product is classified exactly as now. It adds `products[]`: at most 5, most confident first, each with brand, name, variant, form, category and confidence, checked by the same text rules. It also adds `excluded: { otherProducts, unreadable }`. Faces aren't counted (decision E). Anything past 5 is cut in the adapter, and every count is clamped.

```
recognise(placeholder)
├─ photo = repository.findPhoto(item.photoId)
├─ seen = photo.recognition ?? recognizer.recognizeShelf(image)   stored answer wins on retry
│   └─ errors: as today, on the placeholder only; no siblings are made
├─ products empty, exactly 1 unreadable → needs-retake (#55)   decision A
├─ products empty, otherwise → rejected + counts; photo deleted decision A
└─ products ≥ 1:
   ├─ collapse identical (brand|name|variant|form)
   ├─ room = min(5, 20 − itemsInScan + 1)          over → skipped_over_limit
   ├─ allowance = remaining(user) ∧ remaining(scan.ip_hash)  rest → skipped_daily_limit
   ├─ repository.fanOut(photo, placeholderMove, siblings, counts)  ONE D1 batch:
   │     1. UPDATE scan_photos SET recognition, counts, state='recognised'
   │        WHERE state='recognising'
   │     2. INSERT sibling … ON CONFLICT(scan_id, position) DO NOTHING, guarded by count(*) < 20
   │     3. guarded UPDATE placeholder FROM 'recognising' (today's transition write)
   ├─ enqueue next: placeholder + siblings in matching  fails → undo placeholder;
   │                                                    siblings left to the reaper
   ├─ push needs-you once if any needs-confirmation  key is per scan, so no extra pushes
   └─ publish photo, then publish each item (with photoId)
```

- **Slot positions (proposed):** `position = photo.position × 5 + rank`. The placeholder is rank 0. A slot is the same on every retry, so `ON CONFLICT DO NOTHING` makes the fan-out idempotent and two photos' jobs can never collide (§1 item 5). Order is by photo, then rank. This departs from the issue's "position = creation order".
- **No early `done`.** The siblings exist in the same batch that moves the placeholder, so `publish`'s check that every item is final (:828) always sees them.
- **A retake skips the fan-out.** An item with its own `photo_key` takes only `products[0]`.

AS-DESIGNED One shelf photo with three products, one of them doubtful.



4.3 Confirm and the image an item shows

AS-BUILT

```
POST …/items/{itemId}/confirmation → ScanService.confirm   scans.service.ts:275
└─ transition needs-confirmation → matching (+ enqueue match)  :279–291

itemView(item)                                              :852
├─ product = ready ∧ formulationId ? catalog.productView(f) : — :853
├─ image = stock (product.imageUrl)                          :854–857  ready only
│        | user photo via 5-min signed link (photo_key)      :858–861  any state with a key
│        | null
```

AS-DESIGNED

```
confirm: unchanged per item; siblings are untouched   guarded by its own state
         + releasePhotoIfUnneeded(item.photoId)   see 4.6

itemView(item, stock)
├─ image = stock  from catalog.stockImagesOf(productIds)  (proposed)
│          one batched read per view, any state with a product_id
│        | own photo_key (retake) | shared photo's key while held  signed link
│        | null → client shows brand + name   the normal case at confirmation (§1 item 1)
```

A `needs-confirmation` item shows the **whole shelf photo**, which is what story 21 asks for, with its own brand and name. The stock image appears from `match` on, and only if the catalog already holds one.

4.4 Back photo, match, research, analyse

AS-BUILT

`addBackPhoto` (:301–331), `match` (:602), `research` (:646; not-found → `needs-back-photo` at :666), `readLabel` (:672), `labelUnreadable` with `maxBackPhotos = 3` (:727–736) and `analyse` (:738) all work on one item. The only shared data they read is the catalog's.

AS-DESIGNED

No change per item. The one addition is that every `transition` of an item with a `photo_id` ends with `releasePhotoIfUnneeded`. Research and analysis are already shared through the catalog, so two items of the same product in different photos pay for research once (catalog.service.ts:179–212, `ResearchInProgressError` wait at scans.service.ts:656–658).

4.5 Retake (#55 part 1, in flight)

IN FLIGHT · PR #61

```
recognise → unreadable | label-not-visible
└─ photos.delete(photoKey); tries < 3 ? needs-retake : needs-identification
POST …/items/{itemId}/retake → ScanService.retake
└─ put under the item's own key; needs-retake → recognising
   records a scan_retakes row for the quota
```

#61 is based on a staging that predates #53: it drops `form` from `recognised`, and its migration 0013 clashes with the 0013–0015 already merged.

AS-DESIGNED

- An item that has products from a shared photo never enters `needs-retake`: its read was fine. Only a placeholder can, when its photo produced no products and exactly one unreadable product (decision A).
- A retake sets the item's own `photo_key` and leaves `photo_id` as it was, for grouping. Recognising a retake is **single-product**: it takes `products[0]` and doesn't fan out.
- Story 15 ("retake one product when its read was wrong") is dropped (decision B). A wrong read is fixed by confirm or the #72 typed name.

4.6 When a photo is deleted

AS-BUILT

Photo	Deleted when	Where
Front, rejected at recognise	Before the row says `rejected`	scans.service.ts:579–582
Front, any other outcome	**Never**, until the guest cleanup or account deletion	:724–725; scans.cleaner.ts:21–25
Back photo	After it's read, or when unreadable	:707–720
Upload whose save failed	Immediately	:193–195

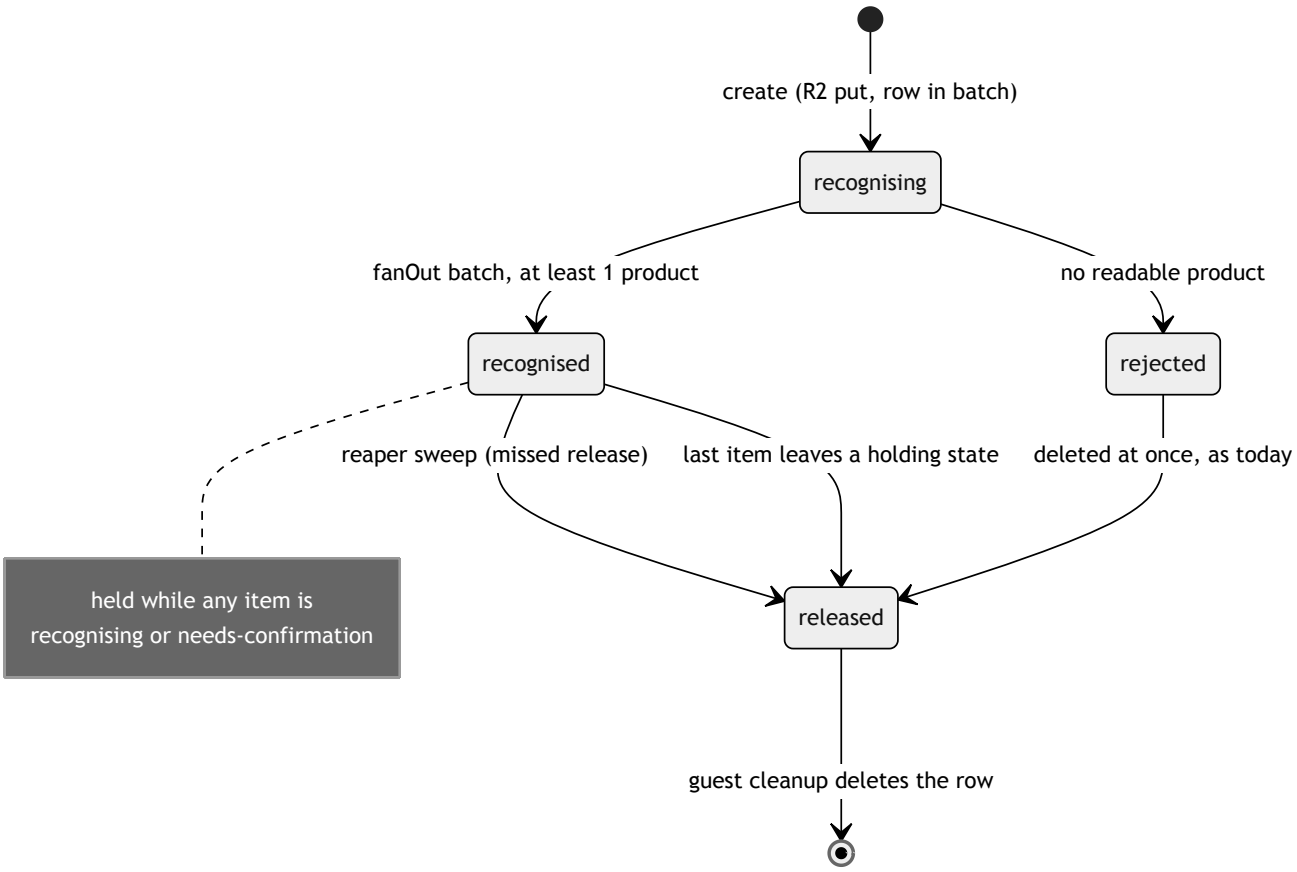
AS-DESIGNED

`releasePhotoIfUnneeded(photoId)` (proposed). A photo is *held* while any of its items is in `recognising` or `needs-confirmation`, plus #55's `needs-retake` and `needs-identification` if they show the photo. Once no item is in a holding state, no item can return to one, so release is one-way and safe to repeat:

1. Read the photo and check that none of its items holds it.
2. `photos.delete(photo_key)`, deleting from R2 first, as the rejection path does today.
3. `UPDATE scan_photos SET photo_key = NULL WHERE id = ? AND photo_key IS NOT NULL`, then publish `photo`.

The reaper adds a sweep for photos still holding a key with no holding item (story 34). This applies to single-photo scans as well, which is the behaviour change in §1 item 2.

AS-DESIGNED Lifecycle of a **scan_photos** row (proposed).



4.7 Live updates: `GET …/{scanId}/events`

AS-BUILT

```
ScanService.watch(scanId)                                  scans.service.ts:229
├─ ownedScan → scanView(scan, items)  → events.subscribe  :230–231
│    └─ DurableObjectScanEvents.subscribe → ScanStream.fetch
│       durable-object.scan-events.ts:16; scan-stream.ts:40
└─ catchUp(scanId, items)  :245–269
   re-read; send each item whose state/updatedAt changed; done if all final
ScanStream.publish(event): done → end all; else frame('item', event.item)  scan-stream.ts:61–68
```

AS-DESIGNED

```
ScanEvent = item { item & photoId } | photo { id, position, state, skipped } | done
ScanSnapshot = { id, createdAt, photos[] }     same nested shape as the GET response
ScanStream.publish: done → end; photo → frame('photo', …); item → frame('item', …)
catchUp: compares photos (state, skipped, key held) as well as items
```

4.8 Daily allowance

AS-BUILT

The count is **item rows created** in the last 24 h, plus back-photo uploads, per user and per IP hash (scans.repository.ts:140–155, 215–227). It's checked at `create` (N photos) and at `addBackPhoto` (1) (scans.service.ts:182, 308). The route text says "30 photos per user and 100 per address" (scans.routes.ts:24).

AS-DESIGNED

No change to the query. Siblings are item rows, so they're counted automatically, which makes the allowance "products" without changing the count. What's new is the check inside `recognise`: `remaining = min(30 − user count, 100 − IP count)`, using the scan's `ip_hash`, and only the extra products are cut. Two photos recognised at the same moment can overshoot by a few. That's the same race `create` already documents (:170–171). The route text changes to "products".

4.9 Reaper and guest cleanup

AS-BUILT

`reap` re-queues working items older than 15 min and fails them after 2 h. It repairs a lost `done` (scans.service.ts:393–414, cron every 10 min, scheduled.ts:8). The cleanup deletes the item and back-photo keys, then the rows (scans.cleaner.ts:21–25; scans.repository.ts:163–213).

AS-DESIGNED

`reap` also covers siblings whose enqueue failed after the fan-out batch: they're in `matching`, a working state, so the existing sweep re-queues them after 15 min. It also adds the missed-release sweep. `photoKeysForUsers` adds `scan_photos.photo_key`, and `deleteForUsers` deletes `scan_photos` rows before `scans`.

5. Failure modes

Condition	TODAY	AFTER
Recognizer returns more than 5 products	n/a (one product)	Adapter keeps the 5 most confident; the rest go to `skipped_over_limit`
Scan already has 20 items	n/a (≤10 items)	Sibling INSERT guard skips them; counted as over-limit
Daily allowance runs out mid-photo	Whole upload refused with 429 at `create`	Placeholder kept (already counted); extras skipped as `daily-limit`, most confident kept first

No readable beauty product	Placeholder `rejected`, photo deleted (:579–582)	Rejected with the photo's counts, unless exactly one unreadable product was seen: then `needs_retake` (#55)
`AiBudgetExceededError` at recognise	Item `failed/research_busy`	Placeholder fails the same way; no siblings created; photo released
`AnthropicRequestError`	`failed/unavailable`, permanent (no retry)	Same, on the placeholder
Retry after the fan-out batch applied	n/a; `earlierRead` restores one product	Stored `recognition` reused; slot conflicts mean no duplicates
Next-step enqueue fails after the batch	Move undone, error rethrown, queue retries (:803–811)	Placeholder undone and retried; siblings already in `matching` are re-queued by the reaper after 15 min
Two products in one photo are the same	n/a	Collapsed before insert (story 9)
Same new product in two photos of one scan	n/a	Two items; research paid once (catalog wait); out of scope to merge
Photo release: R2 delete fails	n/a	Row keeps its key; next transition or reaper sweep retries
Ready product with no stock image	Shows the user's photo (kept for ever)	`image: null`; app shows brand and name (§1 item 2)
Client on the old flat shape	n/a	Breaks; accepted pre-production

6. What stays the same

- The upload route, its limits, multipart body and the per-photo checks (scans.routes.ts:16–36; scans.service.ts:884–897).
- The per-item state machine and every step after `recognise`: match, research, back photo, read label, analyse, the 0.6 confirmation threshold and `maxBackPhotos = 3`.
- The guarded-update pattern, job keys `itemId:step:at` (scans.job-specs.ts:19) and how `done` is decided.
- Catalog identity, matching, research and analysis, apart from the one batched stock-image read.
- Push keys: "needs you" stays one per scan per need, so five doubtful products still send one push.
- No crops, no bounding boxes, no image transforms. The recognize model stays the configured Haiku until the eval says otherwise.

7. Delivery sequence

This starts only after #55's four PRs merge. They need rebasing first: #61 to #73 carry migrations 0013 to 0016, which clash with the merged 0013 to 0015, and #61 predates `form`. Each step below is a stacked PR that is green on its own.

1. **Schema and repository.** `scan_photos`, `photo_id`, the migration, contract tests (fake and D1), and upload writes photos. Behaviour is unchanged: items read their image through the photo.
2. **Recognizer.** `recognizeShelf`, the prompt and schema, the memory fake, and the report-only shelf eval (photos from the owner, never committed). The service still takes product 0.
3. **Fan-out.** Collapsing duplicates, slots, the 5 and 20 caps, the daily-limit cut, skipped counts, the idempotent batch, and the service tests listed in #74.
4. **Photo lifecycle.** `releasePhotoIfUnneeded`, the reaper sweep, the cleaner, and the image fallback change.

5. **API and live updates.** The nested response, `stockImagesOf`, the `photo` event, the DO branch, the snapshot and `catchUp`, and `npm run openapi`.

8. Decisions (settled 2026-10-18)

	Question	Decision
A	With #55, does a shelf photo with no readable product go to `needs_retake`, or is it rejected?	Rejected with counts, **unless** no product was readable and exactly one unreadable product was seen. Then it's `needs_retake`, so #61's retake still serves a blurry single product.
B	Story 15: is retaking a wrong read in scope?	Dropped. Confirm and the #72 typed name fix a wrong read without another photo.
C	Is it fine that a ready product with no stock image loses the user's photo?	Yes. `image: null`, and the app shows brand and name.
D	Slot positions instead of creation order?	Yes. `position = photo.position × 5 + rank`.
E	Count faces as a skip reason?	No. Faces aren't counted or reported.